

INSTITUTO INFNET
ENGENHARIA DA COMPUTAÇÃO



RELATÓRIO TÉCNICO FINAL
SISTEMAS DIGITAIS EMBARCADOS

**Integração ARM64 + FPGA com processamento MAC, telemetria e saída
HDMI**

Aluno: João Paulo Fonseca de Araújo

E-mail: joao.paraujo@al.infnet.edu.br

Projeto de Bloco: Sistemas Digitais Embarcados

[26E2_5]

Rio de Janeiro
2026

RESUMO

Este relatório revisa e amplia a documentação do projeto de Sistemas Digitais Embarcados desenvolvido ao longo dos TPs 1 a 5. A solução utiliza um programa Assembly AArch64 executado em QEMU para gerar oito pacotes de dez bytes protegidos por CRC-8 e uma arquitetura Verilog que recebe bytes de uma UART simulada, confere integridade e campos do protocolo, executa comandos aceitos e transmite respostas ACK/NACK ao testbench.

A versão inicial empregava majoritariamente pacotes conhecidos, previamente validados por um script. A revisão introduziu geradores independentes de testes pseudoaleatórios e de corrupção sistemática de bits, permitindo transmitir ao parser também dados intencionalmente inválidos, sem reparo prévio. Foram demonstrados 1.000 casos pseudoaleatórios aprovados (167 comandos aceitos e 833 rejeições esperadas), testes aprovados dos 256 valores de byte UART e da recuperação após stop bit inválido. O parser também foi modificado para tratar framing_error e timeout de pacote parcial; o teste de recuperação passou a registrar três comandos válidos e dois erros de recepção sem necessidade de reset.

A bateria sistemática de CRC foi preparada para 2.630 casos, mas seu resultado integral ainda não foi apresentado. Na placa Tang Nano 4K, o **top_final.v** mantém a entrada física por botões e permite observar o executor de comandos, MAC em Q8.8, telemetria em BSRAM, LED e quatro padrões HDMI. **Não foi implementada comunicação UART elétrica entre ARM64 e FPGA:** a ida e volta serial ocorre integralmente na co-simulação, com resposta ao testbench, não ao processo ARM64.

Palavras-chave: AArch64; FPGA; Tang Nano 4K; Verilog; UART; CRC-8; robustez; testes pseudoaleatórios; timeout; framing_error; MAC; HDMI.

SUMÁRIO

RESUMO	2
1 INTRODUÇÃO	4
2 OBJETIVOS	5
3 EVOLUÇÃO DO PROJETO: TP1 AO TP5	6
4 ARQUITETURA FINAL DO SISTEMA	7
5 IMPLEMENTAÇÃO ARM64 E GERAÇÃO DOS PACOTES	8
5.2 Inspeção dos bytes com QEMU e GDB	9
6 CO-SIMULAÇÃO E VALIDAÇÃO DO PROTOCOLO	10
6.5 Ampliação da metodologia de testes	14
6.7 Testes pseudoaleatórios de pacotes	15
6.8 Bateria sistemática de CRC	15
6.9 Estresse e recuperação UART	16
7 IMPLEMENTAÇÃO DO DATAPATH FPGA	19
8 SÍNTESE, PLACE & ROUTE E TIMING	20
9 VALIDAÇÃO FÍSICA NA TANG NANO 4K	25
10 RESULTADOS E DISCUSSÃO	28
11 CONCLUSÃO	30
REFERÊNCIAS TÉCNICAS E FONTES DE EVIDÊNCIA	31

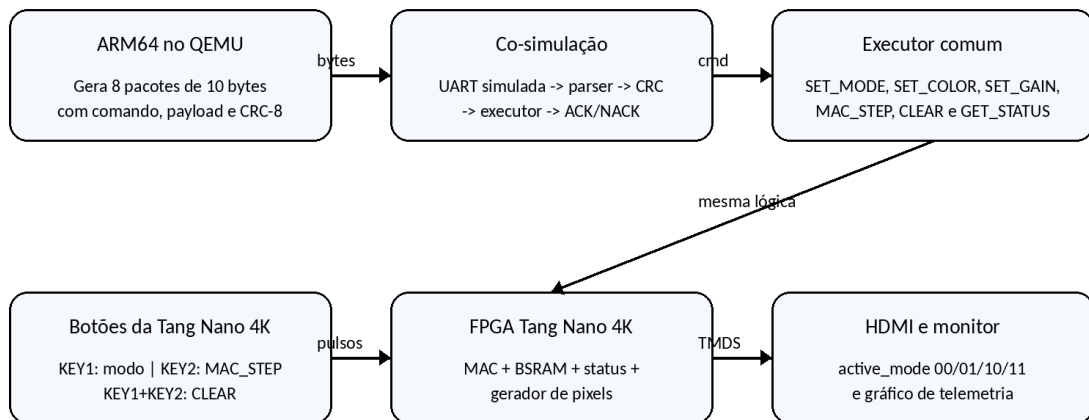
1 INTRODUÇÃO

O projeto final de Sistemas Digitais Embarcados representa a consolidação de uma sequência de protótipos desenvolvidos ao longo de cinco TPs. A proposta inicial partiu de um controlador de vídeo no qual um processador ARM produziria dados de pixel e uma FPGA cuidaria da temporização e da exibição. Ao longo das entregas, as limitações de memória da Tang Nano 4K, a indisponibilidade do Raspberry Pi Zero 2W físico e a necessidade de demonstrar processamento, comunicação e observabilidade conduziram a uma arquitetura final diferente da concepção inicial, porém preservando o objetivo central de cooperação entre software ARM64 e lógica reconfigurável.

Na solução final, o lado ARM64 é executado no QEMU e gera comandos estruturados. Esses bytes alimentam uma co-simulação Verilog na qual uma UART simulada transporta o pacote, um parser valida os campos e o CRC, o executor aplica comandos e uma resposta ACK ou NACK é produzida. Em paralelo, a implementação física na Tang Nano 4K utiliza os dois botões onboard como fonte local de comandos equivalentes, de modo que MAC, telemetria, LED e HDMI possam ser demonstrados sem afirmar a existência de um enlace serial físico que não foi montado.

A saída HDMI é utilizada como interface de observação do estado interno. O valor `active_mode` possui quatro combinações: 00 exibe barras de cores; 01 exibe um padrão xadrez; 10 exibe um gradiente RGB; e 11 apresenta a telemetria da unidade MAC. O LED onboard foi reservado para status de atividade e erro. Assim, a imagem exibida não constitui apenas um efeito visual: ela torna observável o estado de uma arquitetura digital composta por entrada, controle, processamento, memória e saída.

Arquitetura final do projeto ARM64 + FPGA



Integração ARM64-FPGA: validada por co-simulação Hardware: validado fisicamente na Tang Nano 4K

Figura 1 - Arquitetura conceitual da solução final.

Fonte: elaboração própria a partir das evidências do projeto (2026).

2 OBJETIVOS

2.1 Objetivo geral

Consolidar os cinco TPs em um protótipo integrado capaz de demonstrar, de forma reproduzível, a interação conceitual entre software AArch64 e lógica FPGA, incluindo protocolo de comandos com detecção de erros, processamento MAC em ponto fixo, armazenamento de telemetria e visualização por HDMI na Tang Nano 4K.

2.2 Objetivos específicos

- Gerar, em Assembly AArch64 executado no QEMU, uma sequência determinística de oito pacotes de comando.
- Validar os pacotes por CRC-8 e disponibilizá-los à co-simulação em formato binário e hexadecimal.
- Receber bytes por uma UART simulada, detectar CRC inválido, comando inválido, tamanho inválido e sequência duplicada.
- Executar comandos de controle e processamento por meio de um executor comum às validações lógica e física.
- Implementar uma unidade MAC 16 x 16 bits com acumulador de 32 bits, resultado Q8.8 e indicação de overflow.
- Armazenar resultados recentes em BSRAM para formar uma base de telemetria.

- Gerar vídeo HDMI estável e associar os quatro valores de active_mode a padrões visuais distintos.
- Utilizar o LED onboard para sinalizar comando aceito, erro de protocolo e erro grave.
- Confirmar síntese, place and route, recursos dedicados, pinagem, timing e programação física da Tang Nano 4K.

3 EVOLUÇÃO DO PROJETO: TP1 AO TP5

3.1 TP1 - arquitetura inicial ARM-FPGA-VGA

O TP1 definiu a arquitetura preliminar de um controlador VGA com framebuffer ARM→FPGA. O Raspberry Pi Zero 2W seria responsável por calcular conteúdo de tela e enviá-lo por SPI, enquanto a Tang Nano 4K armazenaria os pixels em BRAM e geraria sincronismos VGA em 640 x 480 @ 60 Hz. Nessa etapa foram introduzidos os conceitos de endereço de pixel, framebuffer, geração de cores e divisão de responsabilidades entre CPU e FPGA.

3.2 TP2 - sincronismo de vídeo, padrões e fluxo de build

No TP2, a arquitetura foi concretizada por módulos Verilog de sincronismo e geração de padrões, testbench com Icarus Verilog e inspeção de waveforms no GTKWave. O lado ARM evoluiu para rotinas com loops, condicionais e escrita em buffer. A comunicação SPI ainda era tratada como etapa planejada ou emulada, e o uso de VGA permaneceu como referência de temporização de vídeo.

3.3 TP3 - máquina de estados, entradas físicas e QEMU

O TP3 introduziu uma FSM receptora de comandos, sincronizador de dois flip-flops, debounce de aproximadamente 20 ms e reset de inicialização. A indisponibilidade do Raspberry Pi Zero 2W físico levou a frente ARM64 a ser validada por QEMU, GDB e objdump. Também foram estudados GPIO mapeados em memória, loops, condicionais, tabela de salto e parser de comandos. Essa etapa estabeleceu a separação que seria mantida até o projeto final: FPGA validada fisicamente e ARM64 validado por emulação.

3.4 TP4 - MAC, BRAM, DSP e telemetria

No TP4, o sistema ganhou processamento numérico efetivo. Foi implementada uma unidade MAC com multiplicador 16 x 16 bits e acumulador saturado de 32 bits, buscando a inferência de DSP dedicado. Memórias em bloco foram utilizadas para coeficientes e amostras, e o relatório de síntese confirmou o uso de recursos DSP e BSRAM. No lado ARM

foram acrescentadas aritmética multi-palavra, conversões numéricas, lookup tables, operações bitwise e NEON SIMD. Também surgiu um mecanismo de comunicação simulada com checksum e registro de telemetria.

3.5 TP5 - protocolo, biblioteca Assembly e integração

O TP5 consolidou o protótipo em torno de uma biblioteca Assembly modular e de um protocolo com envio, recepção, checksum, end-of-frame, handshaking e telemetria. O MAC foi revisado com casos de borda adicionais, e a arquitetura FPGA continuou sendo validada na Tang Nano 4K. O enlace elétrico ARM-FPGA permaneceu pendente, mas a lógica de protocolo e o processamento de ambos os lados estavam maduros para a consolidação final.

3.6 Consolidação no projeto final

O projeto final substituiu o framebuffer completo por geração procedural de pixels e migrou a demonstração física para HDMI, utilizando a base oficial da Sipeed. O protocolo foi formalizado em pacotes de dez bytes com CRC-8; os bytes produzidos pelo QEMU passaram a alimentar diretamente um testbench que simula a UART; e a Tang Nano passou a demonstrar localmente o mesmo executor por meio dos botões. Essa decisão fecha o ciclo dos TPs sem mascarar a ausência do canal físico ARM-FPGA.

4 ARQUITETURA FINAL DO SISTEMA

4.1 Divisão das responsabilidades

Subsistema	Responsabilidade na solução final
ARM64/QEMU	Montar os oito comandos, calcular CRC-8 e escrever 80 bytes no fluxo de saída.
Bridge Python	Converter commands.bin para commands.hex e validar a estrutura dos pacotes.
Co-simulação Verilog	Reproduzir o transporte UART, parser, validação, execução e respostas.
Tang Nano 4K	Executar controle, MAC, BSRAM, status, geração procedural de pixels e HDMI.
Botões onboard	Gerar comandos locais equivalentes no protótipo físico.
LED onboard	Indicar atividade, erro de protocolo/comando e erro grave.
Monitor HDMI	Exibir active_mode e a telemetria do MAC.

4.2 Protocolo de comando

Cada comando possui dez bytes. O primeiro byte 0xA5 marca o início do pacote. Em seguida são enviados versão, número de sequência, identificador do comando, tamanho do

payload, quatro bytes de payload e um CRC-8 com polinômio 0x07 calculado sobre os bytes 1 a 8. O número de sequência permite rejeitar retransmissões acidentais de um comando já processado.

Byte	Campo	Função
0	SOF = 0xA5	Marca o início do pacote.
1	Versão = 0x01	Identifica a versão do protocolo.
2	Sequência	Distingue comandos e permite detectar duplicatas.
3	Comando	SET_MODE, SET_COLOR, SET_GAIN, MAC_STEP, CLEAR, GET_STATUS ou PING.
4	Length	Quantidade de bytes úteis no payload, de 0 a 4.
5-8	Payload	Parâmetros do comando; bytes não usados permanecem em zero.
9	CRC-8	Detecta corrupção acidental do pacote.

4.3 Caminho físico e caminho de co-simulação

A integração ARM64-FPGA possui dois caminhos de validação complementares. Na co-simulação, os bytes produzidos pelo QEMU entram em uma UART Verilog e percorrem o parser, o executor e o construtor de resposta. No hardware físico, os botões da placa geram comandos locais e alimentam o executor, permitindo observar a mesma lógica de controle, MAC, BSRAM, LED e HDMI. Portanto, o trabalho demonstra integração funcional entre as partes, mas não uma transmissão UART elétrica em tempo real entre QEMU e a placa.

5 IMPLEMENTAÇÃO ARM64 E GERAÇÃO DOS PACOTES

5.1 Build, execução no QEMU e verificação

O fluxo final é automatizado por Makefiles. O assembler aarch64-linux-gnu-as produz os objetos de main.s, packet.s e syscalls.s; o linker gera packet_gen; o qemu-aarch64 executa o binário e redireciona a saída para commands.bin. Em seguida, bin_to_hex.py cria commands.hex e check_packets.py valida cada pacote, incluindo SOF, versão e CRC.

5.2 Inspeção dos bytes com QEMU e GDB

A geração do arquivo **commands.bin** foi complementada com uma inspeção do **packet_buffer** em memória. Nessa demonstração, o QEMU executa o binário AArch64 com depuração remota; o GDB interrompe a execução na rotina **write_all**, permitindo observar os dez bytes que já foram montados pelo **packet.s**, antes de sua escrita na saída padrão.

```
Terminal 1: qemu-aarch64 -g 1234 arm64/build/packet_gen > /tmp/commands.bin
Terminal 2: gdb-multiarch arm64/build/packet_gen
(gdb) target remote :1234
(gdb) break write_all
(gdb) continue
(gdb) x/10xb &packet_buffer
```

A primeira parada pode exibir o comando **PING** (A5 01 00 7F 04 11 22 33 44 26); após continuar, a sequência seguinte apresenta **SET_MODE(1)** (A5 01 01 01 01 01 00 00 00 91). O procedimento comprova os valores preparados pelo programa ARM64; não implica uma transferência elétrica para a placa.

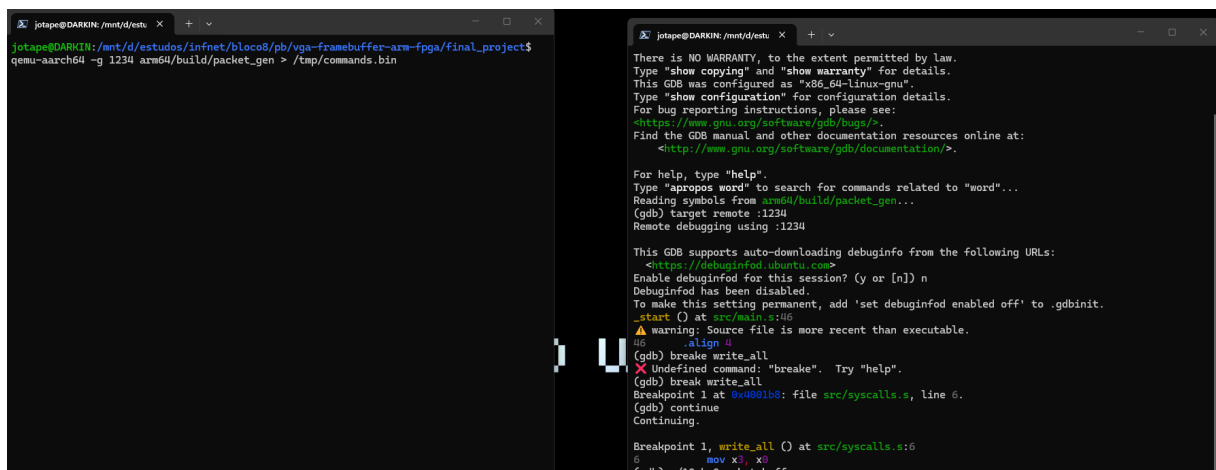


Figura 24 - Inspeção dos pacotes PING e SET_MODE em memória com QEMU/GDB.

Fonte: evidência de execução no WSL/GTKWave (2026).

5.3 Origem dos estímulos e separação dos fluxos

Foram estabelecidas duas origens distintas de dados para a co-simulação: **(a)** os oito comandos determinísticos gerados pelo executável ARM64 no QEMU; e **(b)** as sequências de teste produzidas diretamente por scripts Python. O **check_packets.py** valida o arquivo do ARM64, mas não corrige, filtra ou pré-aprova os pacotes adversariais usados nos novos testbenches.

Os bytes dos arquivos **.hex** são reproduzidos como bits UART pelo testbench Verilog. O circuito sob teste executa suas próprias validações; o resultado esperado é guardado separadamente pelo testbench e somente é utilizado para comparar as respostas. Não há envio de ACK/NACK de volta para o binário ARM64, pois ele não contém um receptor de respostas UART.

gerados pelo ARM64 para fechar a cadeia arquivo -> UART simulada -> parser -> executor -> resposta.



Figura 4 - PASS do tb_mac_unit.

Fonte: elaboração própria (2026).



Figura 5 - PASS do tb_led_status.

Fonte: elaboração própria (2026).

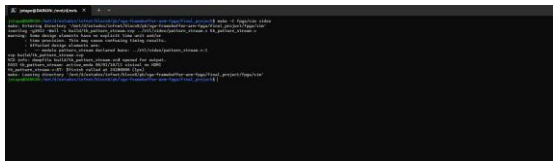


Figura 6 - PASS do tb_pattern_stream.

Fonte: elaboração própria (2026).

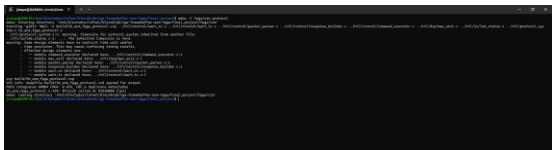


Figura 7 - PASS da integração ARM64-FPGA.

Fonte: elaboração própria (2026).

O resultado final da integração registra oito ACKs, um erro de CRC e uma sequência duplicada detectados. Essa combinação é importante porque testa tanto o caminho nominal quanto situações de falha, evitando uma validação baseada apenas em comandos corretos.

6.2 Pacote válido e ACK

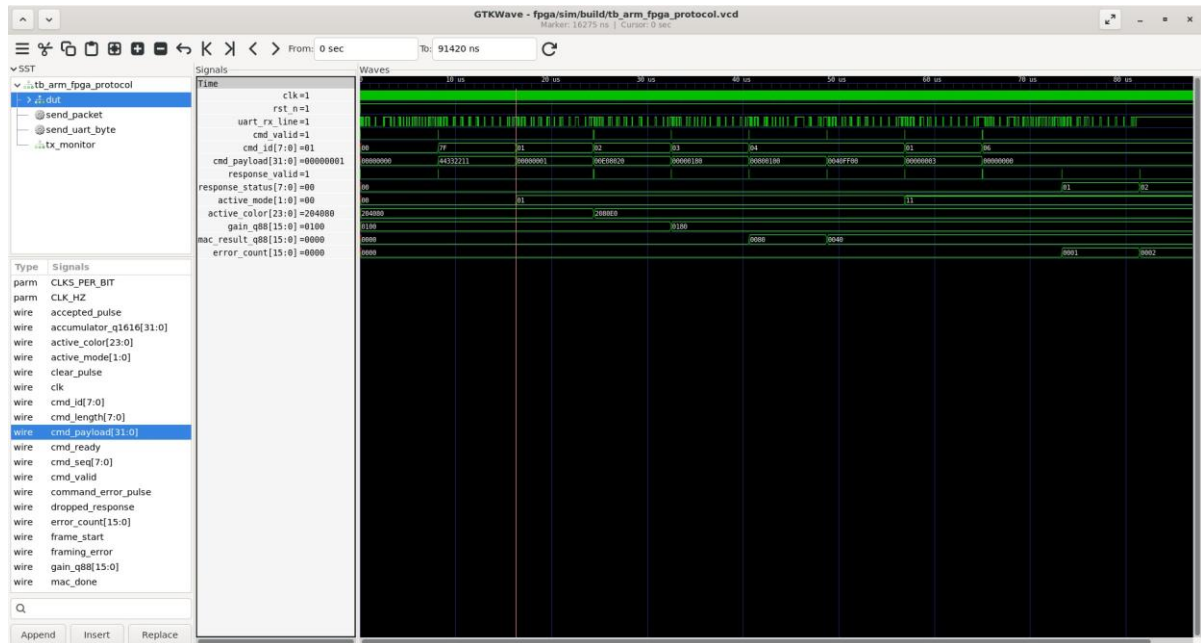


Figura 8 - Waveform da integração mostrando pacote válido, resposta ACK e atualização de estado.

Fonte: elaboração própria a partir das evidências do projeto (2026).

Na região de pacote válido, cmd_valid é gerado e a resposta utiliza status 0, correspondente a ACK. O estado de controle pode ser atualizado após a aceitação do comando, confirmando que a validação do protocolo precede a execução.

6.3 Detecção de CRC inválido

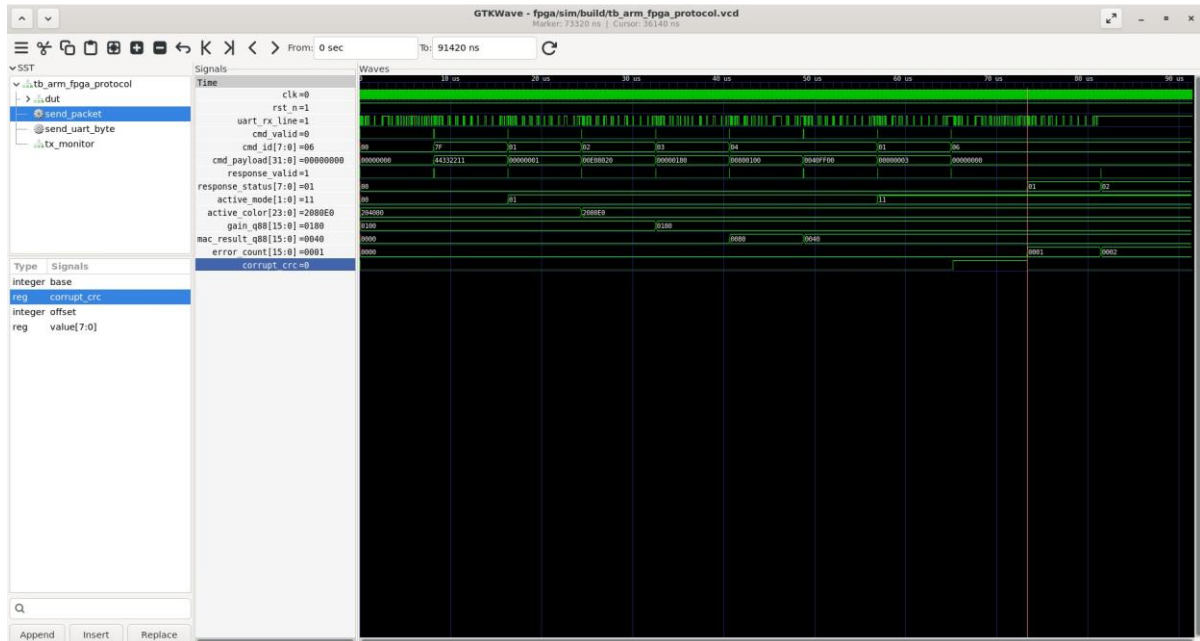


Figura 9 - Waveform do pacote com CRC propositalmente corrompido, status 01 e incremento do contador de erros.

Fonte: elaboração própria a partir das evidências do projeto (2026).

O testbench altera o último byte do pacote para produzir um CRC inválido. A resposta assume status 01 e error_count passa de 0000 para 0001. O comando não é aceito como uma transação válida, demonstrando que a integridade do pacote é verificada antes da execução.

6.4 Detecção de sequência duplicada

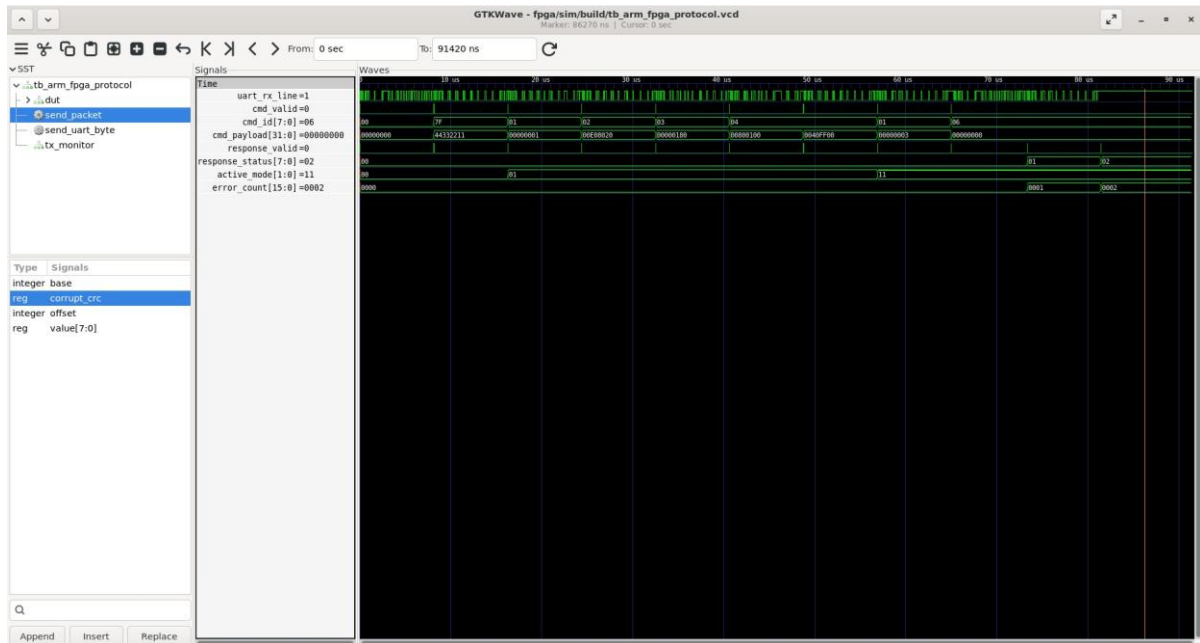


Figura 10 - Waveform de sequência duplicada, status 02 e error_count igual a 0002.

Fonte: elaboração própria a partir das evidências do projeto (2026).

Após o erro de CRC, o testbench retransmite um pacote cuja sequência já havia sido processada. O parser retorna status 02, eleva o contador total de erros para 0002 e não gera uma nova execução do comando. Esse mecanismo impede efeitos duplicados causados por retransmissões.

7 IMPLEMENTAÇÃO DO DATAPATH FPGA

7.1 Entrada física, sincronização e debounce

Os sinais dos botões são assíncronos em relação ao clock interno e também estão sujeitos ao bounce mecânico. Por isso, cada botão passa por sincronização e debounce antes de gerar pulsos de comando. KEY1 percorre os modos de vídeo; KEY2 gera MAC_STEP; e o acionamento conjunto produz CLEAR. Essa lógica permite demonstrar o datapath físico mesmo sem UART externa.

7.2 Unidade MAC em Q8.8

A unidade MAC recebe dois operandos signed de 16 bits em formato Q8.8, multiplica os valores e soma o produto a um acumulador de 32 bits. O acumulador preserva maior faixa numérica durante a soma e o resultado é disponibilizado novamente em Q8.8. O sinal overflow identifica saturação ou condição grave para a lógica de status.

6.5 Ampliação da metodologia de testes do protocolo

A campanha original utilizava sobretudo os oito pacotes gerados pelo ARM64, previamente conferidos pelo script **check_packets.py**, com dois cenários adicionais de falha: CRC incorreto e número de sequência repetido. Essa verificação já exercitava o CRC no Verilog, mas oferecia baixa diversidade de condições adversas.

Na revisão, os novos geradores passaram a fornecer diretamente aos testbenches conjuntos de pacotes válidos e intencionalmente inválidos. Esses pacotes **não são reparados ou descartados pelo Python antes do envio**. O módulo **packet_parser.v** recebe apenas os bytes; confere CRC-8, versão, identificador, tamanho e sequência; e decide se o comando deve ser liberado ao executor. O testbench compara sua decisão a um resultado esperado independente.

6.6 Caminho de ida e volta da UART na co-simulação

Ida: gerador Python ou arquivo do ARM64 → arquivos binários/hexadecimais → testbench → serialização UART 8N1 → **uart_rx.v** → **packet_parser.v** → **command_executor.v** (quando aceito).

Volta: decisão de validação do parser → **response_builder.v** → **uart_tx.v** → recepção/decodificação pelo **testbench**, que verifica código de status, sequência e integridade da resposta. O programa AArch64 no QEMU **não** recebe essa resposta e o top físico **top_final.v** **não** incorpora atualmente a UART.

Etapa	Arquivo/módulo	Responsabilidade
Estímulos	gen_random_packets.py / gen_crc_stress.py	Criar cenários; gravar pacotes de entrada e resultados esperados.
Transmissão	tb_random_protocol.v / tb_crc_stress.v	Ler os pacotes e simular os bits de uma UART.
Recepção	uart_rx.v	Detectar start/stop, reconstruir os bytes e indicar framing_error.
Validação	packet_parser.v	Validar CRC/campos, sinalizar ACK/NACK ou descartar pacotes incompletos.
Resposta	response_builder.v / uart_tx.v	Montar e serializar resposta de oito bytes quando aplicável.
Conferência	testbenches Verilog	Comparar as respostas observadas com as expectativas do teste.

Para pacotes completos que passam pelas verificações, a resposta de oito bytes usa **SOF 0x5A, versão, sequência, status, active_mode, contador de erros (16 bits) e CRC**. O status 0 indica ACK; os status 1, 2, 3 e 4 representam, respectivamente, BAD_CRC, DUP_SEQ,

BAD_CMD e BAD_LEN. Um timeout ou framing error descarta uma transmissão incompleta **sem fabricar um NACK**, uma vez que o pacote pode não possuir sequência confiável.

6.7 Testes pseudoaleatórios controlados de pacotes

O arquivo `bridge/gen_random_packets.py` produz, com semente fixa **12345**, uma massa reproduzível de **1.000 pacotes de dez bytes (10.000 bytes)**. Os casos incluem comandos válidos e seis categorias rotativas de estímulos: válido, CRC inválido, comando desconhecido, comprimento acima de quatro bytes, versão incorreta e sequência duplicada. Os arquivos de entrada e resultados esperados são lidos por `tb_random_protocol.v`, responsável por transmitir os pacotes bit a bit e verificar a resposta do hardware simulado.

```
python3 bridge/gen_random_packets.py
make -C fpga/sim random
# Para inspecionar uma amostra no GTKWave:
cd fpga/sim
vvp build/tb_random_protocol.vvp +cases=20 +wave
gtkwave build/tb_random_protocol.vcd
```

Indicador	Resultado observado
Casos de teste	1.000
Verificações aprovadas	PASS=1.000; falhas=0
Comandos válidos aceitos	167
Pacotes inválidos detectados	833
Origem do resultado	Execução do testbench apresentada anteriormente pelo aluno

Os **833 erros detectados** são estímulos **deliberadamente inválidos**, e não falhas de funcionamento do testbench. A verificação dos 1.000 casos foi demonstrada **antes da última modificação do parser**; por isso, a execução de regressão da bateria completa após o acréscimo do timeout permanece uma confirmação recomendada.

6.8 Bateria sistemática de corrupção de CRC

O arquivo `bridge/gen_crc_stress.py` parte de um pacote-base válido e inverte bits nas oito posições de dados protegidas pelo CRC (bytes 1 a 8) e no próprio byte CRC (posição 9); o marcador SOF não faz parte da cobertura de CRC. O gerador cria **72 alterações de um bit** e todas as **2.556 combinações de dois bits distintos**, além de um pacote válido inicial e um final: **2.630 casos** previstos.

Classe	Quantidade	Resultado esperado
Pacote válido inicial	1	ACK
Um bit invertido	72	BAD_CRC

Classe	Quantidade	Resultado esperado
Dois bits invertidos	2.556	BAD_CRC
Pacote válido final	1	ACK
Total	2.630	2 ACK e 2.628 rejeições previstas

O gerador registra as posições modificadas e os status esperados em arquivos auxiliares; **tb_crc_stress.v** é o responsável por transmitir a sequência na UART simulada e confrontar as respostas. A primeira e a última transação válida verificam se o receptor segue operacional após uma longa sequência de rejeições. **O resultado integral de execução dessa bateria não foi apresentado**; portanto, a tabela expressa o planejamento do ensaio e não um PASS comprovado.

```
python3 bridge/gen_crc_stress.py
make -C fpga/sim crc-stress
```

6.9 Estresse do receptor UART: tb_uart_stress.v

O ensaio **tb_uart_stress.v** estimula diretamente o **uart_rx.v**. Ele verifica a recepção dos **256 valores possíveis de byte** (0x00 a 0xFF), a rejeição de um pulso de falso start, a detecção de um stop bit inválido e a capacidade de receber outro byte após o erro. Isso testa o enquadramento serial, independentemente da lógica de CRC do pacote.

```
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/final_project$ make -C fpga/sim uart-stress
make: Entering directory '/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/final_project/fpga/sim'
iverilog -g2012 -Wall -o build/tb_uart_stress.vvp ../rtl/control/uart_rx.v tb_uart_stress.v
warning: Some design elements have no explicit time unit and/or
: time precision. This may cause confusing timing results.
: Affected design elements are:
:   -- module uart_rx declared here: ../rtl/control/uart_rx.v:1
vvp build/tb_uart_stress.vvp
VCD info: dumpfile build/tb_uart_stress.vcd opened for output.
PASS UART: 256 valores de byte recebidos
PASS UART: pulso de falso start ignorado
PASS UART: stop bit inválido detectado
PASS UART: recuperou-se do stop bit inválido
PASS tb_uart_stress: bytes_validos=257 framing_errors=1
tb_uart_stress.v:102: $finish called at 506350000 (1ps)
make: Leaving directory '/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/final_project/fpga/sim'
```

Figura 25 - Execução aprovada dos testes de varredura de bytes, falso start, stop inválido e recuperação.

Fonte: evidência de execução no WSL/GTKWave (2026).

O terminal registrou **PASS UART: 256 valores de byte recebidos**, falso start ignorado, stop bit inválido detectado e recuperação aprovada; o resumo indicou **bytes_validos=257** e **framing_errors=1**. O byte adicional é a transação válida enviada após o erro, utilizada para confirmar a recuperação do receptor.

6.10 Falha reproduzida e correção de recuperação: tb_packet_recovery.v

O ensaio **tb_packet_recovery.v** força interrupções no nível de pacote: envia PING completo, transmite somente uma fração de outro pacote e aguarda; em seguida, envia novo pacote válido; depois induz erro de stop bit durante a recepção e verifica uma nova aceitação. Antes da correção, a execução terminava em **FAIL timeout**, pois a lógica de parsing continuava aguardando bytes e não incrementava o contador de erros de recepção.

```

: -- module packet_parser declared here: ../rtl/control/packet_parser.v:1
: -- module response_builder declared here: ../rtl/control/response_builder.v:1
: -- module uart_rx declared here: ../rtl/control/uart_rx.v:1
: -- module uart_tx declared here: ../rtl/control/uart_tx.v:1
vvp build/tb_packet_recovery.vvp
VCD info: dumpfile build/tb_packet_recovery.vcd opened for output.
PASS controle: PING completo recebido
FATAL: tb_packet_recovery.v:153: FAIL timeout: ack=1 cmd=1 respostas_bytes=8 erros=0 (esperado 1)
Time: 47390000 Scope: tb_packet_recovery
make: *** [Makefile:40: packet-recovery] Error 1
make: Leaving directory '/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/final_project/fpga/sim'
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/final_project$

```

Figura 26 - Teste de recuperação antes do ajuste: timeout esperado não detectado.

Fonte: evidência de execução no WSL/GTKWave (2026).

A correção acrescentou o registrador **idle_count** e o parâmetro **PACKET_TIMEOUT_CYCLES** ao **packet_parser.v**. Quando o limite sem novos bytes é atingido, o estado parcial é descartado, o contador de erros aumenta e o parser retorna à busca de SOF 0xA5. Também foi adicionada uma entrada **framing_error** e sua ligação em **protocol_system.v**; o stop bit inválido agora aborta a transação parcial de modo imediato.

Na configuração dos testes, o limite foi definido por **CLKS_PER_BIT × 40**: equivalente a quatro durações de byte UART 8N1 entre recepções. Os dois casos representam **erros de recepção sem ACK/NACK**, não comandos negados por CRC. Essa distinção evita contabilizar respostas inexistentes.

```

vvp build/tb_packet_recovery.vvp
VCD info: dumpfile build/tb_packet_recovery.vcd opened for output.
PASS controle: PING completo recebido
PASS timeout: pacote parcial foi descartado sem ACK/NACK
PASS recuperacao: pacote valido apos interrupcao
PASS framing error: pacote abortado
PASS recuperacao UART: pacote valido apos stop invalido
PASS tb_packet_recovery: ACK=3, comandos=3, erros_recepcao=2, respostas_bytes=24
tb_packet_recovery.v:177: $finish called at 121660000 (1ps)
make: Leaving directory '/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/final_project/fpga/sim'
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/final_project$

```

Figura 27 - Teste de recuperação após a correção, incluindo timeout e framing_error.

Fonte: evidência de execução no WSL/GTKWave (2026).

Verificação	Resultado observado
PING completo de controle	PASS
Descarte por timeout sem ACK/NACK	PASS
Pacote válido depois da interrupção	PASS
Aborto por stop bit inválido	PASS
Pacote válido após erro de enquadramento	PASS
Resumo	ACK=3; comandos=3; erros_recepcao=2; respostas_bytes=24

Os três ACKs correspondem às três transações completas aceitas; o total de **24 bytes** corresponde às três respostas de oito bytes. Os **dois erros de recepção** são o timeout e o framing_error, situações em que o parser descarta o pacote sem solicitar execução do comando.

6.11 Observação no GTKWave e rastreabilidade

A validação temporal pode ser inspecionada nos arquivos **tb_arm_fpga_protocol.vcd**, **tb_random_protocol.vcd**, **tb_uart_stress.vcd** e **tb_packet_recovery.vcd**. Os principais sinais são **byte_data**, **byte_valid**, **receiving**, **index**, **idle_count**, **crc**, **framing_error**, **cmd_valid**, **response_valid**, **response_status** e **error_count**.

As novas evidências ampliam a confiança nos mecanismos implementados, mas não demonstram prova formal de correção, cobertura de todos os valores possíveis de payload, transmissão ARM64 ↔ FPGA em hardware ou ausência de falhas de interoperabilidade em condições de campo.

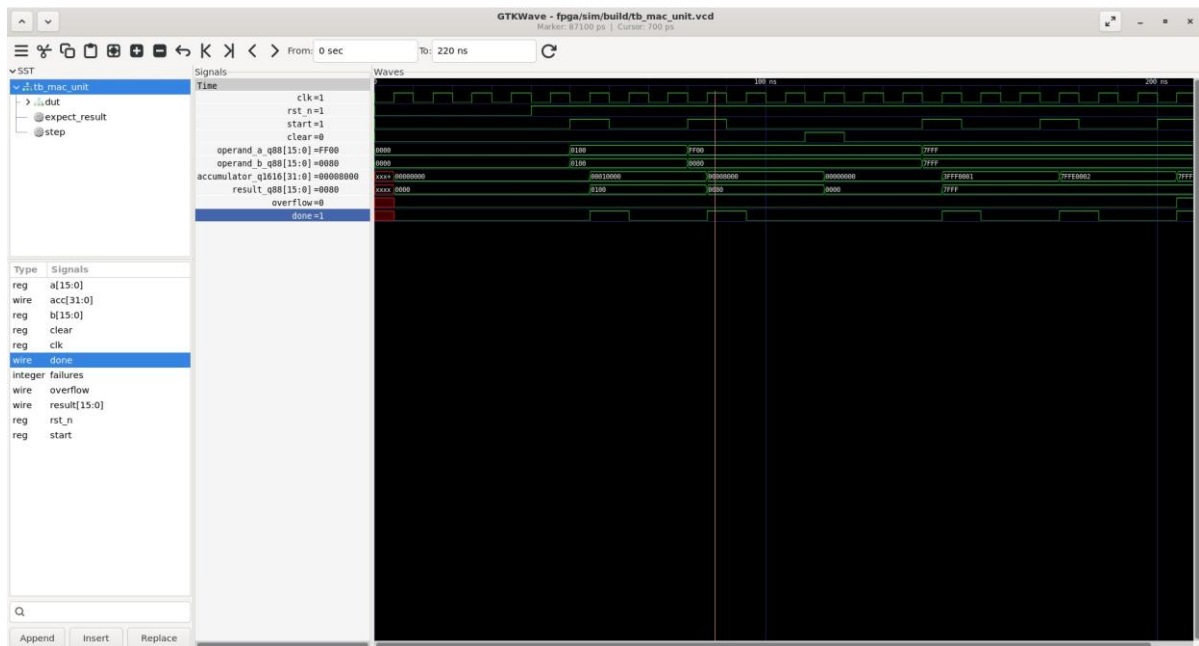


Figura 11 - Waveform da unidade MAC mostrando start, operandos, acumulador, resultado, overflow e done.

Fonte: elaboração própria a partir das evidências do projeto (2026).

A waveform evidencia operações sucessivas. No ponto destacado, os operandos FF00 (-1,0 em Q8.8) e 0080 (+0,5) produzem atualização coerente do acumulador e result_q88 igual a 0080 (+0,5), considerando o resultado acumulado da operação anterior. O pulso done marca a conclusão de cada passo.

7.3 BSRAM de telemetria

A memória de telemetria armazena resultados concluídos pelo MAC e fornece amostras ao gerador de vídeo. Em vez de reservar memória para um framebuffer completo, a solução final calcula os pixels proceduralmente e dedica a BSRAM ao histórico numérico. Essa decisão reduz drasticamente o consumo de memória e permite transformar resultados internos em um gráfico HDMI.

7.4 LED onboard como canal de status

O LED onboard foi desacoplado da representação de active_mode. Um comando aceito gera uma piscada curta; um erro de protocolo/comando gera duas piscadas; e um erro grave mantém uma piscada rápida contínua até CLEAR. A validação temporal dessa lógica é registrada no testbench.

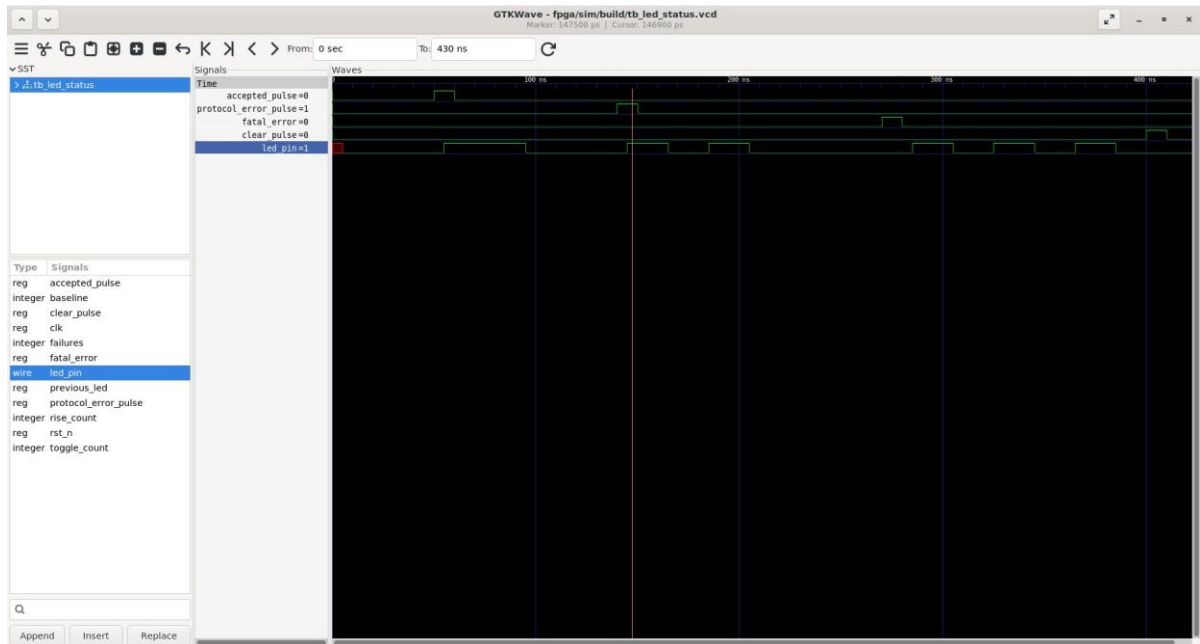


Figura 12 - Waveform de led_status com eventos de comando aceito, erro de protocolo, erro grave, clear e saída led_pin.

Fonte: elaboração própria a partir das evidências do projeto (2026).

7.5 Geração HDMI e active_mode

O pipeline de vídeo reutiliza blocos oficiais SVO/HDMI da Sipeed e insere pattern_stream como fonte dos pixels. active_mode seleciona barras, xadrez, gradiente ou telemetria. Dois pequenos indicadores no canto superior direito representam os bits mode[1] e mode[0], permitindo verificar visualmente o estado binário do sistema. A geração procedural evita a necessidade de armazenar um quadro 640 x 480 completo.

8 SÍNTESE, PLACE & ROUTE E TIMING

8.1 Projeto final no Gowin

O projeto físico foi montado sobre a base HDMI oficial, mantendo os módulos SVO, PLLVR e CLKDIV. top_final foi selecionado como top module, e as constraints finais substituíram as constraints do exemplo original. protocol_system, UART, parser e testbenches permanecem no fluxo de co-simulação e não integram o top físico atual.

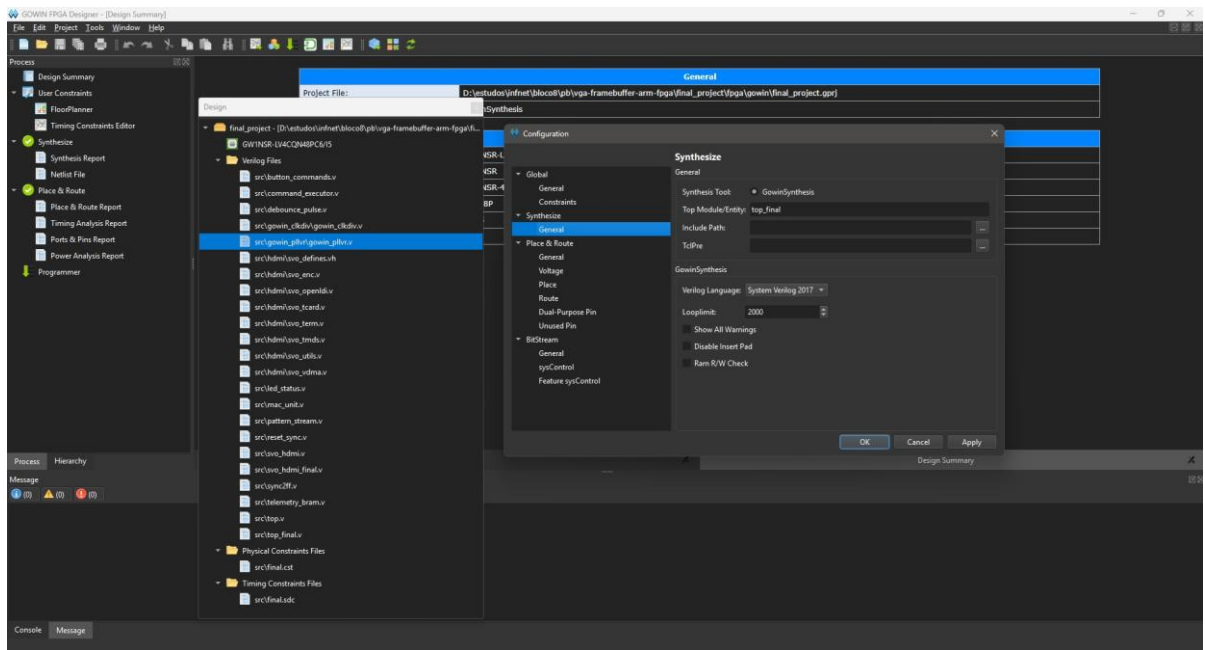


Figura 13 - Árvore do projeto Gowin e configuração de top_final como módulo principal.

Fonte: elaboração própria a partir das evidências do projeto (2026).

8.2 Síntese e utilização de recursos

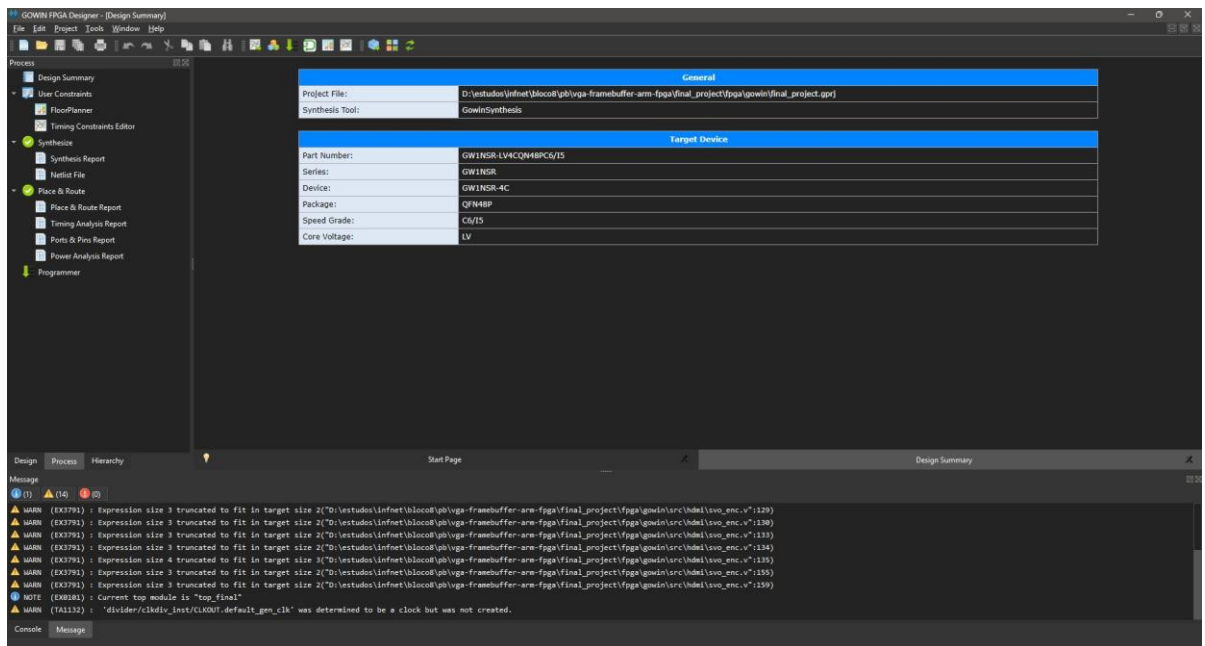


Figura 14 - Design Summary após síntese e Place & Route, sem erros e com top module top_final.

Fonte: elaboração própria a partir das evidências do projeto (2026).

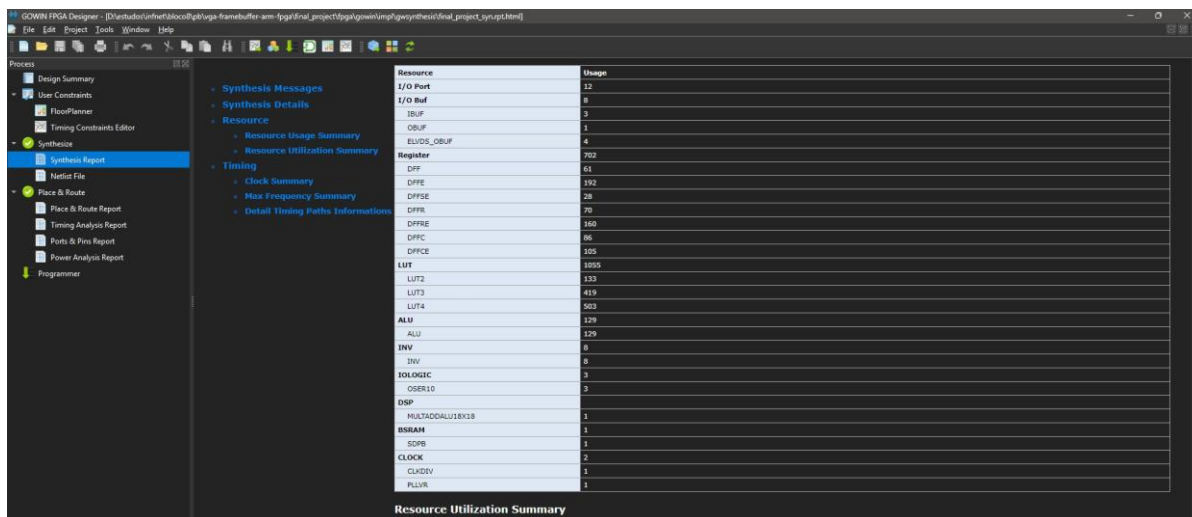


Figura 15 - Resource Utilization Summary do projeto final.

Fonte: elaboração própria a partir das evidências do projeto (2026).

A síntese confirmou recursos dedicados coerentes com a arquitetura. Foram utilizados 1 bloco DSP MULTADDALU18X18, 1 BSRAM do tipo SDPB, 1 PLLVR, 1 CLKDIV, 3 OSER10 e 4 ELVDS_OBUF, além de 1055 LUTs e 702 registradores. Esses dados comprovam que o MAC foi mapeado em hardware aritmético dedicado, a telemetria utiliza memória em bloco e o pipeline HDMI emprega os recursos de clock, serialização e saída diferencial esperados.

Recurso	Uso observado	Interpretação
MULTADDALU18X18	1	DSP dedicado utilizado pelo produto da unidade MAC.
BSRAM / SDPB	1	Memória em bloco utilizada pela telemetria.
PLLVR	1	Geração do clock rápido do pipeline HDMI.
CLKDIV	1	Derivação do clock de pixels de 25,2 MHz.
OSER10	3	Serialização dos canais de dados TMDS.
ELVDS_OBUF	4	Três pares de dados HDMI e um par de clock.
LUT	1055	Controle, comparações e lógica combinacional.
Register	702	Estados, contadores, pipelines e registradores de controle.

8.3 Análise temporal



Figura 16 - Resumo de clocks, frequência máxima e Total Negative Slack após implementação.

Fonte: elaboração própria a partir das evidências do projeto (2026).

O clock externo foi reconhecido em 27 MHz. A PLL gera 126 MHz, e o divisor produz 25,2 MHz para o domínio de pixels. Para esse domínio, o relatório apresenta restrição de 25,2 MHz e Fmax estimada em 35,252 MHz. O Total Negative Slack Summary registra TNS = 0,000 e zero endpoints negativos para os domínios listados. Em capturas detalhadas do relatório de timing, os caminhos de setup e hold observados apresentaram slack positivo, reforçando a ausência de violação crítica no conjunto analisado.

8.4 Pinagem e constraints físicas

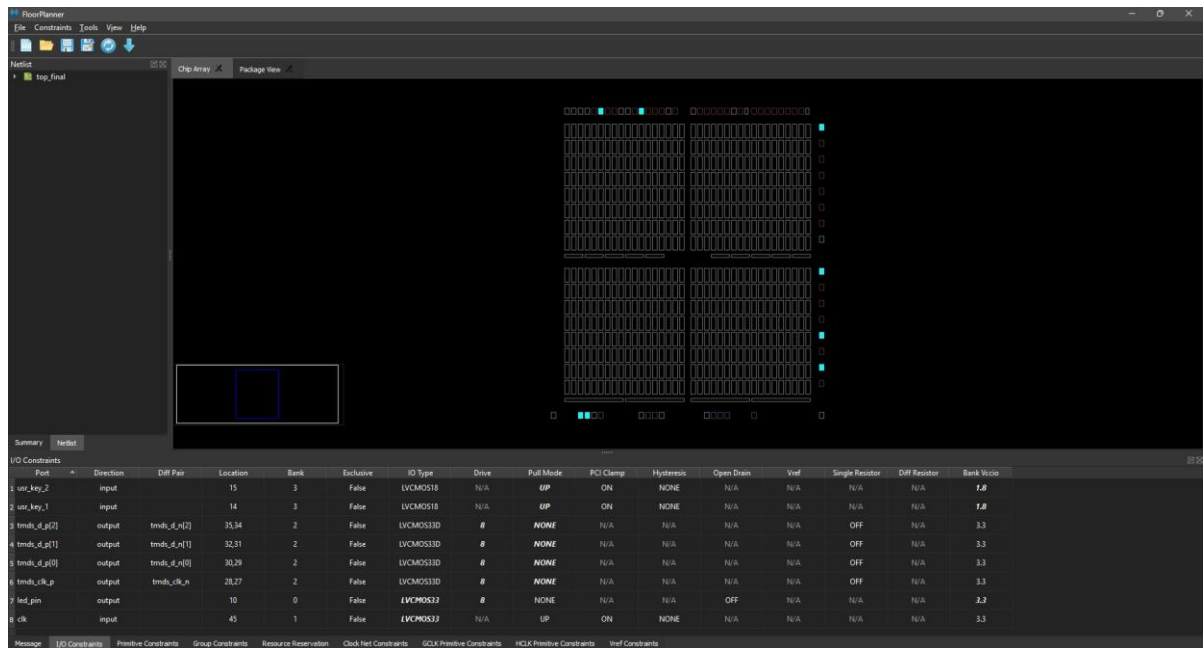


Figura 17 - FloorPlanner com pinagem final de clock, botões, LED e pares HDMI.

Fonte: elaboração própria a partir das evidências do projeto (2026).

A pinagem final associa clk ao pino 45, usr_key_1 ao pino 14, usr_key_2 ao pino 15 e led_pin ao pino 10. Os pares HDMI ocupam 30/29, 32/31, 35/34 para os três canais de dados e 28/27 para o clock diferencial. A captura também confirma bancos e níveis de I/O utilizados na implementação.

8.5 Programação em SRAM

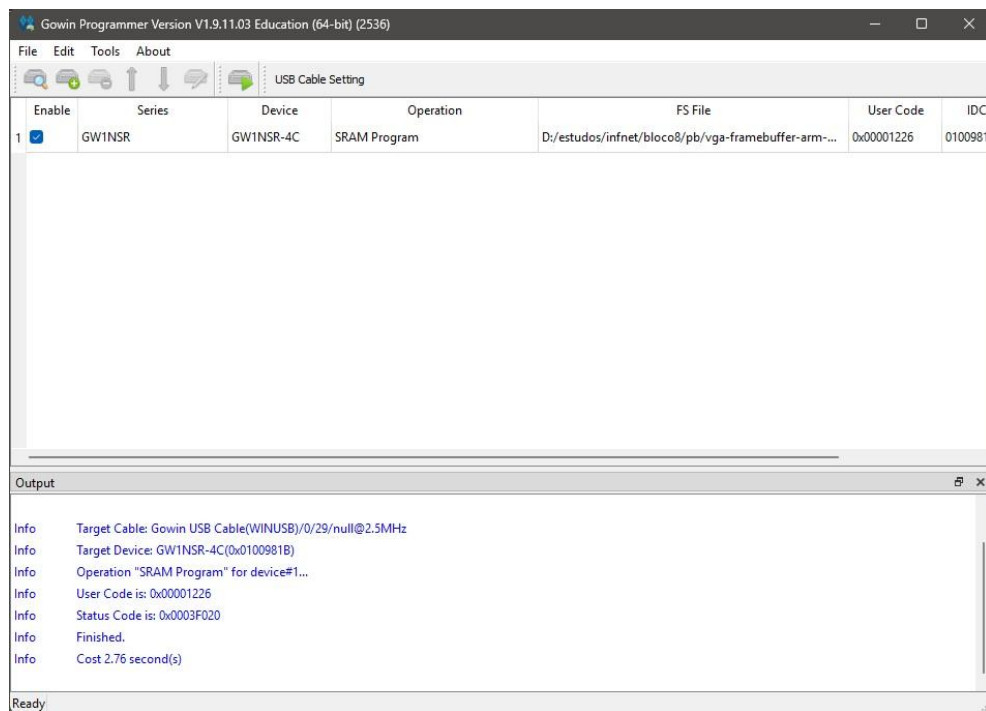


Figura 18 - Gowin Programmer após programação SRAM concluída com sucesso.

Fonte: elaboração própria a partir das evidências do projeto (2026).

O bitstream final foi carregado na Tang Nano 4K utilizando SRAM Program. A mensagem Finished confirma a conclusão da gravação temporária utilizada para a validação física.

9 VALIDAÇÃO FÍSICA NA TANG NANO 4K

9.1 active_mode 00 e 01



Figura 19 - active_mode 00: barras de cores.

Fonte: elaboração própria (2026).

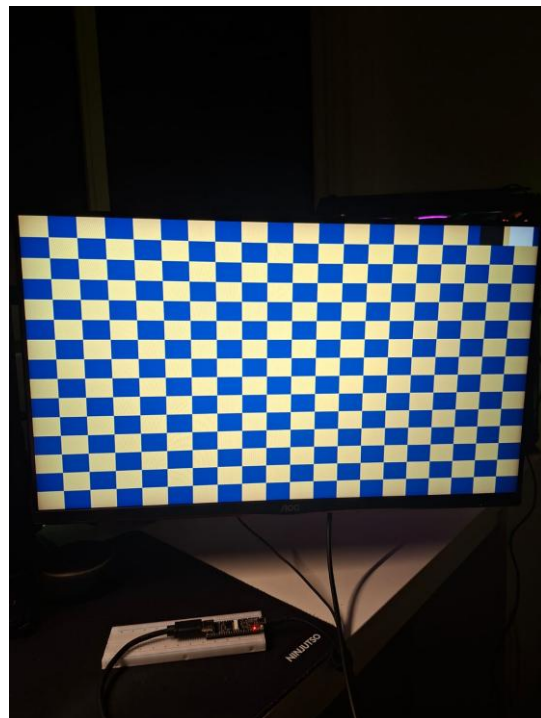


Figura 20 - active_mode 01: padrão xadrez.

Fonte: elaboração própria (2026).

As duas primeiras telas confirmam a troca de estado por comando local e a geração contínua de vídeo. No modo 00, o monitor recebe barras verticais; no modo 01, o padrão muda para xadrez. Os indicadores binários no canto superior direito acompanham o valor de active_mode.

9.2 active_mode 10

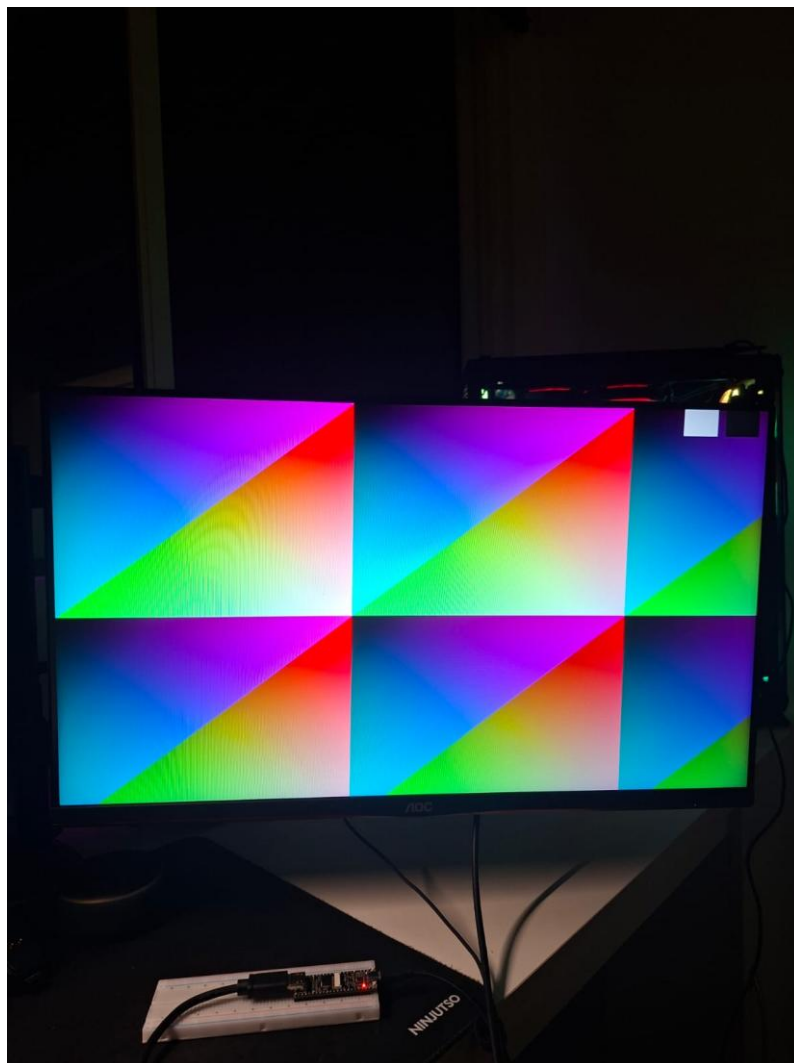


Figura 21 - active_mode 10: padrão de gradiente RGB exibido fisicamente no monitor.

Fonte: elaboração própria a partir das evidências do projeto (2026).

9.3 active_mode 11 e telemetria

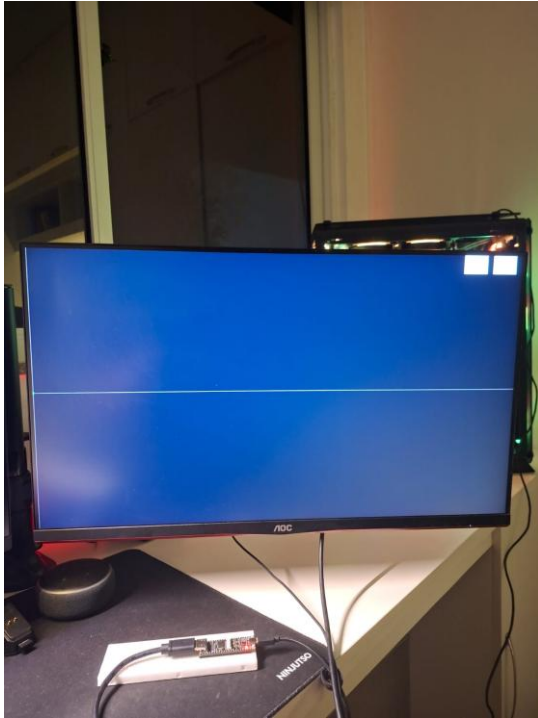


Figura 22 - active_mode 11 antes de novas amostras MAC.

Fonte: elaboração própria (2026).

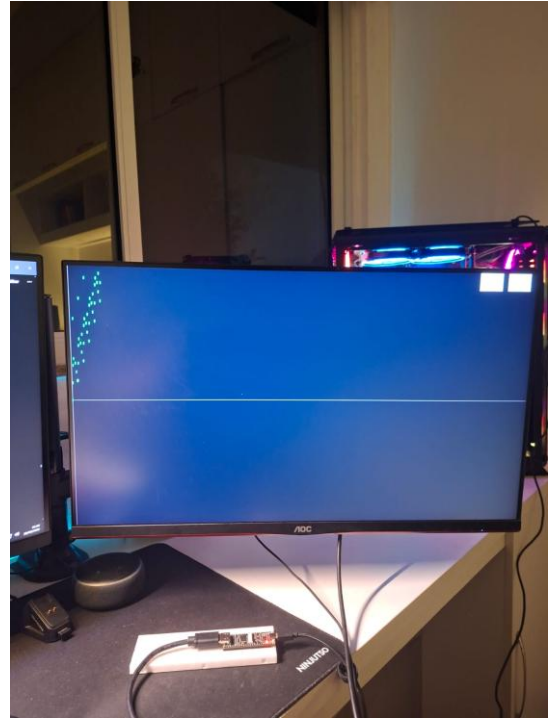


Figura 23 - active_mode 11 após MAC_STEP, com amostras de telemetria no gráfico.

Fonte: elaboração própria (2026).

O modo 11 utiliza um fundo escuro, uma linha horizontal de referência e pontos derivados das amostras armazenadas. A comparação entre as Figuras 22 e 23 mostra que, após operações MAC, novos pontos passam a ser exibidos na região do gráfico. Essa evidência fecha o caminho físico botão -> comando -> MAC -> BSRAM -> leitura de telemetria -> pattern_stream -> HDMI.

9.4 Observação sobre o LED físico

O funcionamento lógico dos padrões do LED foi comprovado pelo testbench e pela waveform apresentada na Figura 12. As evidências estáticas deste pacote não registram temporalmente uma piscada física, pois uma fotografia isolada não comprova frequência ou quantidade de pulsos. Assim, este relatório atribui a validação dos padrões de status à simulação e não extrapola a evidência para uma medição física temporal do LED.

10 RESULTADOS E DISCUSSÃO

10.1 Síntese consolidada dos resultados

A revisão amplia os resultados documentados na primeira versão do relatório, mantendo intactas as evidências de síntese, place and route, temporização, padrões HDMI e telemetria física. A Tabela a seguir diferencia os testes comprovados por execuções apresentadas dos testes preparados que ainda exigem confirmação.

Item avaliado	Situação	Evidência / observação
ARM64/QEMU (base inicial)	Aprovado no relatório original	8 pacotes corretos; 80 bytes; observação em GDB. A fonte main.s alterada posteriormente requer nova compilação sem erros de comentário.
Integração original por UART simulada	Aprovada no relatório original	8 ACKs; 1 BAD_CRC; 1 DUP_SEQ.
Pacotes pseudoaleatórios	1.000 PASS na execução apresentada	167 aceitos; 833 rejeitados; 0 falhas. Nova regressão após mudança no parser recomendada.
Estresse do receptor UART	Aprovado	256 valores de byte; falso start ignorado; framing_error detectado; recuperação.
Recuperação de pacotes	Aprovado após correção	3 ACKs; 2 erros de recepção; 24 bytes de resposta; timeout e framing_error tratados.
Bateria sistemática de CRC	Preparada; execução integral não comprovada	2.630 casos previstos; 72 flips únicos; 2.556 duplos; 2 bons.
MAC, BSRAM e HDMI	Aprovados conforme evidências originais	tb_mac_unit, tb_pattern_stream, DSP/BSRAM inferidos e 4 modos HDMI físicos.
Síntese, timing e programação física	Aprovados conforme evidências originais	Gowin com TNS 0,000, Fmax 35,252 MHz no domínio citado e programação em SRAM.
UART elétrica ARM64 ↔ FPGA	Não implementada	Ida e volta UART ocorrem no testbench de co-simulação; placa usa botões.

10.2 Comparação entre a versão inicial e os testes revisados

Aspecto	Versão original	Revisão
Pacotes de entrada	8 comandos conhecidos, conferidos antes por <code>check_packets.py</code> ; 2 falhas pontuais no testbench	Adversariais válidos/inválidos: 1.000 exemplos pseudoaleatórios e bateria de corrupção CRC preparada.
CRC no Verilog	Já era calculado e comparado pelo parser	A mesma validação é desafiada por dados propositalmente corrompidos sem correção prévia.
Retorno de resposta	ACK/NACK na UART simulada para o testbench	Permanece no testbench e agora é verificado em mais cenários.
Erros de enquadramento	Deteção pelo receptor UART, sem aborto integrado ao parser	Aborto de pacote parcial e contagem de erro de recepção.
Pacote interrompido	Parser podia permanecer preso em recepção	Timeout por <code>idle_count</code> ; descarte e recuperação sem reset.

A mudança fundamental não é a criação de uma ligação física entre os dois dispositivos: trata-se do **aumento da cobertura experimental do protocolo simulado** e do **endurecimento da máquina de estados do parser**. O Verilog já verificava o CRC anteriormente; a inovação desta revisão foi provocar sistematicamente dados inválidos e demonstrar recuperação após erros de recepção.

10.3 Impacto funcional dos mecanismos de robustez

Na camada de byte, o receptor UART demonstrou ignorar uma transição espúria de start, reconhecer um stop bit inválido e sincronizar o recebimento seguinte. Na camada de pacote, a máquina de estados passou a limitar o tempo de espera entre bytes e a descartar o quadro em caso de `framing_error`. A resposta em simulação é transmitida pela `uart_tx.v` e conferida pelo testbench, porém não há rotina de leitura de retorno no binário ARM64.

O caso mais esclarecedor é a evolução do `tb_packet_recovery.v`: o mesmo ensaio que falhava ao exigir um erro de timeout passou a exibir três comandos aceitos, dois erros de recepção contabilizados e zero falhas nas verificações exercitadas. Os registros de execução e os VCDs são evidências de regressão específica; sua aprovação não substitui a suíte completa de regressão.

10.4 Limitações e pontos ainda não encerrados

As limitações principais continuam sendo: ausência de interface UART elétrica entre uma CPU ARM64 e a FPGA; ausência de recepção ACK/NACK no código AArch64; dependência de estímulos gerados no computador; inexistência de verificação formal exaustiva; e falta de confirmação registrada da bateria sistemática de CRC com 2.630 casos.

Na implementação atual do parser, o comprimento é limitado globalmente a quatro bytes, mas não há confirmação, por este conjunto de evidências, de validação exata do

comprimento exigido por cada comando. Além disso, a integração do mecanismo de respostas sob saturação do transmissor e colisões de múltiplos retornos demanda testes próprios. Essas condições devem permanecer registradas como melhorias futuras, não como funcionalidades já resolvidas.

A adaptação do **packet_parser.v** ao fluxo de **protocol_system.v** foi comprovada pelo teste de recuperação e pela simulação do receptor. Ainda é necessário executar novamente as suítes ARM64, protocolo original, aleatória e CRC com a versão mais recente dos arquivos, evitando confundir resultados históricos aprovados com uma regressão integral já comprovada.

10.5 Reprodutibilidade e evidências

```
make -C arm64 check
make -C fpga/sim all
make -C fpga/sim protocol
python3 bridge/gen_random_packets.py
make -C fpga/sim random
python3 bridge/gen_crc_stress.py
make -C fpga/sim crc-stress
make -C fpga/sim uart-stress
make -C fpga/sim packet-recovery
```

Os arquivos VCD gerados podem ser analisados no GTKWave; os resultados de console devem ser arquivados junto às imagens e aos scripts para rastreabilidade. Um erro de compilação posterior em **arm64/src/main.s**, causado por comentários que utilizavam caractere @ em vez da sintaxe AArch64 apropriada, não constitui falha do protocolo UART, mas precisa ser corrigido antes de afirmar uma nova compilação completa bem-sucedida.

11 CONCLUSÃO

O projeto final consolidou as etapas TP1 a TP5 em duas frentes complementares: co-simulação funcional do protocolo ARM64-Verilog e validação física do processamento/visualização na Tang Nano 4K. No percurso original, o Assembly AArch64 gerou pacotes de dez bytes com CRC-8; o ambiente Verilog reproduziu serialização UART, validação e respostas; a implementação física integrou entradas locais por botões, MAC, BSRAM, LED e quatro modos HDMI.

A revisão de robustez abordou diretamente a crítica de que os dados usados na apresentação inicial eram majoritariamente válidos e previamente conhecidos. Pacotes propositalmente inválidos passaram a ser produzidos por geradores Python e transmitidos, sem reparo prévio, aos módulos UART e parser Verilog. O ensaio pseudoaleatório demonstrou 1.000 verificações aprovadas na execução apresentada, com 167 aceitações e 833 rejeições previstas; os testes de UART confirmaram a recepção dos 256 valores possíveis de byte e o tratamento de um erro de stop bit.

O avanço funcional mais relevante foi a correção de recuperação de pacotes: o parser recebeu tratamento de **timeout entre bytes** e **framing_error**, e o ensaio específico passou a aceitar novos comandos após dois erros de recepção, sem reset. A bateria de estresse de CRC com 2.630 casos foi estruturada e documentada, mas seu resultado final ainda deve ser confirmado para integrar o balanço de testes aprovados.

As respostas ACK/NACK são enviadas pelo **uart_tx.v** exclusivamente ao **testbench** na co-simulação; não retornam ao programa AArch64 e não trafegam por um fio UART até a Tang Nano. Portanto, a revisão fortalece a demonstração da **validação autônoma do protocolo em Verilog** e da **resiliência sob falhas simuladas**, sem extrapolar esse resultado para uma integração elétrica não implementada. Como continuidade, recomenda-se a regressão completa, o refinamento das regras de comprimento por comando, testes de sobrecarga de respostas e, se houver hardware de interface disponível, uma UART física de 3,3 V.

REFERÊNCIAS TÉCNICAS E FONTES DE EVIDÊNCIA

ARAÚJO, João Paulo Fonseca de. Integração ARM64 + FPGA com processamento MAC, telemetria e saída HDMI. Relatório técnico final do Projeto de Bloco, Instituto Infnet, Rio de Janeiro, 2026. Documento-base desta revisão.

PROJETO ARM64-FPGA. Código-fonte: **arm64/src/main.s**, **packet.s**, **syscalls.s**, **fpga/rtl/control/uart_rx.v**, **packet_parser.v**, **fpga/rtl/protocol_system.v** e testbenches em **fpga/sim**. Repositório: github.com/jotaraujo/vga-framebuffer-arm-fpga.

EVIDÊNCIAS DE TESTE. Capturas de terminal, registros da execução no WSL e waveforms VCD apresentados durante a revisão técnica, outubro de 2026; resultados adicionais de CRC integral dependentes de execução e arquivamento.